MY READING TOWN

Technical Demo: Flutter mobile development, architecture, stack, and how it all works

Fernando Pinto Villarroel – 2026

Contents

1. What is My Reading Town?
2. Mobile Development Fundamentals
3. Flutter & Dart
4. Technology Stack
5. Localization
6. Core Game Loop
7. Q&A

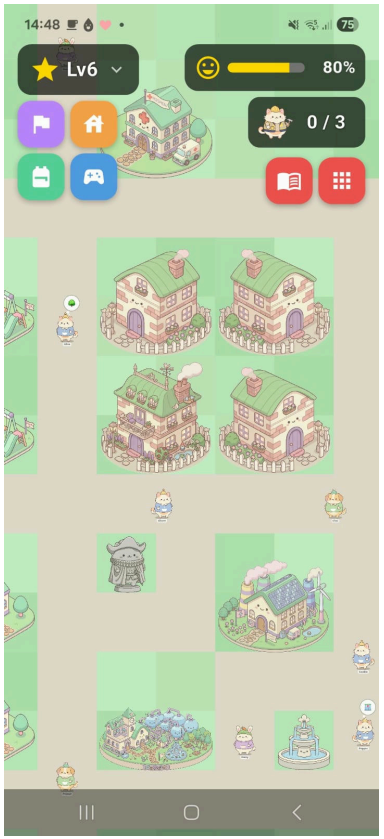
WHAT IS MY READING TOWN?

What is My Reading Town?

A **mobile app** that turns reading into a village-building game.

- Log pages read from any book
- Earn resources: coins, gems, wood, metal
- Spend them to construct buildings and grow a village
- Villagers move in as the village expands
- Missions, minigames, and a guided tour for new players

Goal: make reading a daily, rewarding habit through game mechanics, not a chore.



MOBILE DEVELOPMENT FUNDAMENTALS

Three kinds of software: three mental models

Backend API

Runs on a **server**. No UI. Receives HTTP requests, processes data, returns JSON or HTML.

Users never see this directly.

Node.js, Django, Spring, Go…

Web App

Runs **inside a browser**. HTML + CSS + JS render the UI. The browser is the host environment.

Must be fetched from a server each visit.

React, Vue, Angular, plain HTML…

Mobile App ← us

Runs **natively on device**. Installed once. Has direct access to hardware: camera, GPS, storage, notifications.

No browser. The OS is the host.

Flutter, React Native, Swift, Kotlin…

My Reading Town is a mobile app: it runs offline, persists data on-device, and renders a real-time game. No server, no browser, just Flutter on your phone.

Web app vs mobile app: concrete differences

	Web App	Mobile App (Flutter)
Distribution	URL, browser renders it	Installed package (.apk / .ipa), runs from device storage
Rendering	Browser engine (Blink, WebKit) paints DOM / CSS	Flutter's own rendering canvas (no browser dependency)
Offline	Partial, needs service workers and extra setup	First-class, app works with zero connectivity
Hardware	Limited, sandboxed by browser security model	Full access: camera, file system, notifications, sensors
Performance	JS runtime overhead, reflows, layout recalculations	Compiled to native ARM (60/120 fps game rendering)

	Web App	Mobile App (Flutter)
Updates	Instant, users always get latest when opening URL	Must push a new release through App Store / Play Store

FLUTTER & DART

Flutter: one codebase, any device

Flutter is Google's open-source UI toolkit. Write code **once** in Dart, compile to native for Android, iOS, Web, and Desktop.

How is that possible?

Flutter ships its own **rendering engine**. It does not rely on each platform's native UI components, it draws every pixel itself, directly to the GPU canvas.

No browser. No bridge layer. Flutter compiles to native ARM machine code. Performance is comparable to apps written natively in Swift (iOS) or Kotlin (Android).

Everything in Flutter is a Widget

App Screen

Scaffold (page structure)

AppBarnnavigation

Bodyncontent area

Village Mapn(Flame canvas)

HUD Overlayn(buttons, bars)

Only the subtrees that changed are repainted, not the whole screen.

Flutter's compilation model

Dart source code

Human-readable files, strongly typed and easy to read

→

AOT compilation

Compiled to native ARM64/x64 machine code at build time

→

Native binary

Runs directly on the CPU (no interpreter, no warm-up)

During development: Hot Reload Code changes appear in under a second without restarting, the app state is preserved. Critical for fast iteration.

Compare to web: JavaScript is interpreted in the browser each time. Flutter ships a compiled binary, closer to a native game than a website.

Dart: the language behind Flutter

Dart is Google's strongly-typed, object-oriented language, designed for client-side apps.

- **Statically typed**: errors caught at compile time, not runtime
- **Null safety**: the compiler prevents null-pointer crashes by design
- **Async-first**: concurrency built into the language itself
- **Single-threaded with isolates**: no shared-memory race conditions

Type System

Every value has a declared type. Mistakes are caught before the app runs, not when a user taps a button.

Null Safety

Variables cannot be null unless explicitly allowed. Eliminates an entire class of crashes.

Async / Await

Loading from disk or network never freezes the UI, async operations are a first-class language feature.

Familiar Syntax

Classes, interfaces, generics, lambdas, the same constructs you already know from Java or C#.

TECHNOLOGY STACK

Full tech stack at a glance

UI & Framework

Flutter 3.5+
Dart language
Material Design widgets
Custom kawaii-pastel theme

Game Engine

Flame 1.21
2D isometric grid
Sprite components
Camera & input system

Data & State

sqflite (SQLite)
13 tables, local DB
Provider (state mgmt)
GetIt (DI container)

Platform APIs

Push notifications
Camera & gallery
OS share sheet
File system access

Localization: 5 translation files (EN, ES, FR, IT, PT)

Networking: fetches book metadata from Open Library API

Extras: confetti animations · gallery export · backup & restore

Clean Architecture in Flutter

The codebase is split into **four layers**. Each layer has one responsibility. The key rule: **dependencies only point inward**, outer layers know about inner ones, never the reverse.

Why this matters: the database can be swapped for cloud storage, or the state library replaced, without touching any business logic. The innermost layer has zero framework imports.

This is the same “ports and adapters” pattern used in well-structured backend services, the same principles apply equally well on mobile.

Infrastructure (Flutter UI + Database) screens, widgets, game canvas, SQLite helper

↓ depends on

Adapters (Providers & Repositories) connects Flutter state to business services

↓ depends on

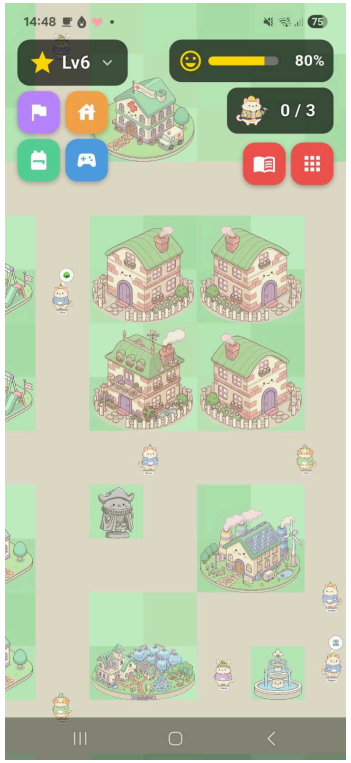
Application (Services) ReadingService, BuildingService, MissionService…

↓ depends on

Domain (Entities + Rules) Book, Villager, Building, pure Dart, no Flutter

Flame: a 2D game engine inside Flutter

Flame is a lightweight game engine built **on top of Flutter**. It gives you a game loop, component system, sprite rendering, and input handling, while still letting regular Flutter widgets share the same screen.



What Flame adds over plain Flutter:

- **Game loop:** runs 60 times per second, driving all animation
- **Component system:** every building, villager, tile is its own object with position, size, and lifecycle

- **Camera:** pan, zoom, and world offset for the village map
- **Input routing:** taps and pinch gestures are forwarded to the right component

The village map is rendered by Flame at 60 fps. The resource bar, modals, and menus on top are regular Flutter widgets, both live on the same screen simultaneously.

Flutter layernHUD, modals, menus

Flame layerngrid, sprites, camera

Data layer: SQLite with sqflite

SQLite is a database engine built into every Android and iOS device. The app talks to it through **sqflite**, a Flutter plugin. No server, the entire database lives as a single file on the device.

Compare to a web app: a web app calls a server, which queries a remote database. Here, the app IS the server, there is no network hop, no latency, no account required.

All data is structured in 13 tables:

Reading tracker

books
reading_sessions
tags / book_tags

Village state

placed_buildings
villagers
road_tiles
unlocked_chunks

Economy

resources
inventory_items
active_powerups

Progression

missions
game_state
minigame_cooldowns

App ⟶ Repository ⟶ SQLite .db file on device

State management: Provider + GetIt

Provider is Flutter's recommended pattern for reactive state. When something changes, only the parts of the screen that care about that data are repainted, not the entire screen.

Three providers drive the app:

- **Village Provider:** buildings, villagers, resources, XP, missions
- **Book Provider:** book collection, reading sessions, search and filters
- **Language Provider:** active language, all translated strings

How a state change flows through the app

User taps "Build" → Provider calls Service

Service saves to SQLite ← Widget rebuilds with new data

Widgets subscribe to providers, they update automatically when the data they watch changes.

LOCALIZATION

5 languages, zero code duplication

The app ships with full translations in **English, Spanish, Portuguese, French, and Italian**. The language can be switched at runtime from the settings screen.

How it works:

- Each language is a text file with key → value pairs
- The language provider loads the right file at startup (or on change)
- Every UI string is looked up by key, the same key works in all 5 languages

Language	"Build" button label
🇬🇧 English	Build
🇪🇸 Spanish	Construir
🇧🇷 Portuguese	Construir
🇫🇷 French	Construire
🇮🇹 Italian	Costruisci

The same pattern applies to every label, button, message, and tooltip across the entire app (hundreds of strings, all in one place per language).

- The chosen language is saved to the local database

Adding a new language only requires adding one text file. No changes to the app logic, no rebuild needed, loaded at runtime.

CORE GAME LOOP

Core game loop: how reading becomes gameplay

1. Log reading

Open a book, enter pages read and time spent

→

2. Earn resources

The app calculates coins, gems, wood, and metal

→

3. Build village

Spend resources to place buildings on the map

→

4. Villagers arrive

Residents move in; happiness rises with more buildings

Resource formula
3 coins per page read
20 gems on book completion
Wood & metal from sessions
50 coins + 20 gems finish bonus

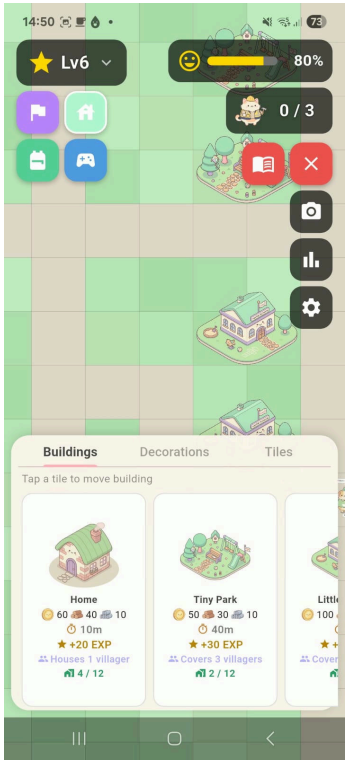
Construction queue
Buildings take real time to complete. A powerup item speeds up construction. Progress is saved across app restarts.

Missions & XP
Completing milestones (build 3 houses, read 500 pages…) awards XP. Missions branch into construction, villager, and reading tracks.

How all layers collaborate at runtime

When a user taps **“Build”** on the village map:

1. **Flame** detects the tap on the grid and fires a callback
2. **GameScreen** (Flutter widget) receives it and opens a bottom sheet
3. User picks a building; the **Village Provider** is called
4. **Building Service** checks the rules (enough resources? valid tile?)
5. The **Building Repository** persists the new building to SQLite
6. The **Resource Repository** deducts costs and saves the updated balance
7. The provider notifies all listening widgets, the HUD counters update
8. The Flame village map places the new building sprite on the grid
9. **Mission Service** checks whether any mission condition is now satisfied



Each step crosses exactly one layer boundary. Business logic never touches the UI; the UI never touches the database.

Analogous backend flow:
HTTP request ⟶ controller ⟶ service ⟶ repository ⟶ database ⟶ response. The same separation of concerns, no network, no HTTP. The “request” is a tap; the “response” is a widget rebuild.

Q&A

Thank you

My Reading Town: tech summary

Flutter 3.5 + Dart · Flame 2D engine · SQLite (sqflite)
Provider state management · GetIt DI · Clean Architecture
5-language i18n · Offline-first · Android & iOS

Open source · Built for the love of reading and game design
Fernando Pinto Villarroel